

CPSC 250

Lecture 1: Primitive Data Types and Memory

Edward Brash

1 Introduction

A common misconception among students entering CPSC 250 is that they should already remember every detail from CPSC 150/150L perfectly. In reality, students enter with a wide range of preparation levels.

The goal of this course is not only to reinforce earlier material, but also to develop a much deeper understanding of how Python actually works internally.

In particular, we will focus heavily on:

- Data structures
- Memory representation
- Algorithmic thinking
- Object-oriented programming
- Numerical data representation

2 Why Data Structures Matter

In many ways, data structures are the central idea in programming.

Every piece of data in a computer program must ultimately be stored somewhere in memory. Understanding:

- how data is represented,
- how it is organized,
- and how efficiently it can be manipulated

is essential for designing good algorithms.

Python intentionally hides many low-level implementation details from programmers. This is convenient, but it can also obscure what is really happening underneath the hood.

In this course, we will repeatedly pull back that abstraction layer.

3 Primitive Data Types in Python

The four basic primitive-style data types we will review are:

1. Integers
2. Floating-point numbers
3. Strings
4. Booleans

3.1 Examples

Type	Examples
Integer	1, 0, 42, -6
Float	1.2, -3.14159, 3.28e-6
String	"a", "Bob", \n
Boolean	True, False

3.2 Important Notes

Different data types:

- require different amounts of memory,
- use different storage mechanisms,
- and support different operations.

4 Integers in Python

One of Python's nicest features is that integers effectively behave as arbitrary-precision numbers.

Unlike C or C++, Python integers do not normally overflow when they exceed a fixed size.

4.1 Overflow in C/C++

In low-level languages like C/C++, integer variables are allocated a fixed number of bytes.

For example:

- 4-byte integers can store only a limited range of values.
- Exceeding that range produces overflow.

Python hides this complexity from the programmer.

5 Python Integers are Objects

Although integers appear to be primitive values, Python integers are actually full Python objects implemented internally in C.

Conceptually:

- a Python integer stores metadata,
- bookkeeping information,
- and dynamically allocated memory.

This flexibility comes at the cost of additional memory overhead.

5.1 Factorial Example

```
1 def factorial(n):
2     if n == 0 or n == 1:
3         return 1
4
5     return n * factorial(n - 1)
6
7 print(factorial(231))
```

Python can compute extremely large integers with hundreds of digits automatically. Internally, Python allocates additional memory as necessary.

5.2 Memory Cost

Even very small integers require substantial overhead in Python.

A Python integer may require roughly 28 bytes or more of memory, depending on the system architecture.

This is because Python integers behave more like objects than primitive machine-level integers.

6 Floating Point Numbers

Most programming languages represent floating point numbers using binary fractions.

This introduces important limitations:

- (a) Some decimal numbers cannot be represented exactly.
- (b) Floating-point arithmetic introduces rounding error.

7 Scientific Notation Review

Recall ordinary scientific notation:

$$3352.28 = 3.35228 \times 10^3$$

This representation separates a number into:

- mantissa/significand
- base
- exponent

8 Binary Fractions

Decimal fractions use powers of 10:

$$32.56 = 3(10^1) + 2(10^0) + 5(10^{-1}) + 6(10^{-2})$$

Binary fractions instead use powers of 2.
For example:

$$1011.011_2$$

corresponds to:

$$1(2^3) + 0(2^2) + 1(2^1) + 1(2^0) + 0(2^{-1}) + 1(2^{-2}) + 1(2^{-3})$$

which equals:

$$8 + 0 + 2 + 1 + 0 + 0.25 + 0.125 = 11.375$$

9 Binary Scientific Notation

Floating point numbers are stored in normalized binary scientific notation.

Example:

$$11.01_2 = 1.101_2 \times 2^1$$

The normalized form always begins with:

$$1.xxxxx$$

This allows one bit to be implicitly assumed rather than explicitly stored.

10 32-Bit Floating Point Representation

IEEE-754 single precision floating point numbers are stored using:

- 1 sign bit
- 8 exponent bits
- 23 mantissa bits

10.1 Exponent Bias

The exponent field uses a bias of 127.

Thus:

$$\text{stored exponent} = E + 127$$

Examples:

$$00000001_2 \rightarrow 2^{-126}$$

$$11111110_2 \rightarrow 2^{127}$$

Special exponent values are reserved for:

- infinity
- NaN (Not a Number)
- denormalized numbers

11 Special Exponent Values

- exponent bits are all 1,
- and the mantissa is all 0,

then the number represents:

$$+\infty$$

or:

$$-\infty$$

depending on the sign bit.

Examples that may generate infinity include:

1 1.0 / 0.0

or arithmetic overflow.

11.0.1 NaN (Not a Number)

If:

- exponent bits are all 1,
- but the mantissa is not all 0,

then the result is NaN.

NaN represents undefined or invalid numerical operations.

Examples include:

```

1 0.0 / 0.0
2 math.sqrt(-1)

```

NaN has unusual comparison behavior:

```

1 float('nan') == float('nan')

```

returns:

```

1 False

```

This behavior is intentional and follows the IEEE-754 standard.

11.1 Summary

Exponent Field	Interpretation
00000000	Denormalized Numbers / Zero
00000001 to 11111110	Normal Floating Point Numbers
11111111	Infinity or NaN

12 Strings in Python

Strings are significantly more complicated internally than they first appear.

Consider:

```

1 s1 = "abcdef"

```

Conceptually, the characters are stored sequentially in memory:

a
b
c
d
e
f

Each character is represented internally using character encoding schemes such as ASCII or Unicode.

12.1 Memory Addresses

```

1 print(s1)
2 print(id(s1))

```

The function `id()` returns the memory identity of the object.

13 Booleans

In Python, booleans are actually a subclass of integers.

Specifically:

`True = 1`

and

`False = 0`

Internally, booleans therefore behave much like integers.

14 Key Takeaways

- Python hides many implementation details.
- Data representation strongly affects algorithm behavior.
- Integers, floats, strings, and booleans are all implemented as objects.
- Floating point numbers are approximations.
- Understanding memory representation is essential for serious programming.